

Offline Evaluation Strategy

Introduction

Offline evaluation lays the groundwork for comparing recommender system approaches before they go live. This document explains the evaluation method used in this project. It sets a clear protocol that all models must adhere to. The method focuses on two main questions: how to divide data without passing on future information to training, and which metrics accurately reflect recommendation quality for our specific case.

Data split strategy

The project uses temporal chronological splitting instead of random splitting. This decision mirrors how recommender systems work in real life. They need to predict future user behavior based on past interactions. Random splits could let the model "see" future ratings during training, which would falsely boost performance estimates.

The splitting process occurs in two steps. First, interactions are sorted by timestamp and split into a combined train-validation set (85%) and a test set (15%). Next, the train-validation portion is divided temporally into training (70% of total) and validation (15% of total). This two-step method makes sure that validation performance shows how well the model generalizes to future time periods, rather than just random samples that were set aside.

The 70/15/15 ratio strikes a balance between having enough training data and ensuring meaningful validation and test sets. Since MovieLens 1M has about one million ratings, this amounts to 700K training interactions and 150K each for validation and testing. The implementation uses `python_chrono_split` from the Microsoft recommenders library, found in `src/data/splitter.py`.

One outcome of temporal splitting is cold-start exposure. In the validation set, 60% of users and 38% in the test set never appeared during training. While this is realistic, it means that evaluation includes both warm-user and cold-user performance together. Models should provide stratified results whenever possible.

Evaluation task and metrics

The main evaluation task is ranking. Given a user, we need to create an ordered list of recommended items. Users see ranked lists rather than predicted rating values, so ranking quality is more important than rating accuracy.

For metric computation, we used Microsoft's `recommenders` library (v1.2.1). This toolkit is proven in production systems and academic benchmarks. Instead of building metrics from the ground up, we rely on established implementations that manage edge cases properly and adhere to known standards. This approach minimizes implementation bugs and makes our results comparable to published research that uses the same library.

The table below summarizes each metric, its purpose, and when it provides useful information.

Metric	Category	Purpose	When Useful
--------	----------	---------	-------------

Metric	Category	Purpose	When Useful
nDCG@k	Ranking (Primary)	Measures ranking quality with position-weighted relevance; normalized to [0,1]	Primary optimization target; comparable across users
MAP@k	Ranking	Average precision across relevant items; emphasizes early hits	When all relevant items matter, not just top positions
Precision@k	Ranking	Fraction of top-k recommendations that are relevant	Simple interpretability; "how many recommendations are good?"
Recall@k	Ranking	Fraction of relevant items appearing in top-k	When coverage of user interests matters
RMSE	Rating	Root mean squared error of predicted vs actual ratings	Diagnosing rating scale calibration; penalizes large errors
MAE	Rating	Mean absolute error of predictions	More robust to outliers than RMSE
Coverage	Beyond-accuracy	Fraction of catalog recommended across all users	Detecting popularity bias; higher means more diverse usage
Novelty	Beyond-accuracy	How obscure recommendations are (inverse popularity)	Measuring long-tail exploration; higher means less mainstream

The main metric is nDCG@k (Normalized Discounted Cumulative Gain). nDCG considers both relevance and position, giving more weight to relevant items that appear higher in the ranking. It compares against the ideal ranking and produces scores between 0 and 1. These scores are comparable across users who have different numbers of relevant items. By default, we evaluate at k=10, which reflects typical recommendation display settings.

Secondary ranking metrics, such as MAP@k, Precision@k, and Recall@k, offer additional perspectives. For diagnostic purposes, RMSE and MAE measure how accurately we predict ratings. While they are not the main optimization goals, they can help determine if a model handles the rating scale correctly, which could shed light on ranking issues. Metrics like Coverage and Novelty assess the diversity of recommendations and help prevent popularity bias.

Scope and limitations

This evaluation setup measures a model's ability to rank relevant items based on past interaction patterns. It looks at offline prediction quality, assuming that previous behavior indicates future preferences.

However, the setup misses several key aspects. User behavior online is different from offline evaluation because recommendations affect what users engage with next, creating feedback loops that offline metrics cannot see. Long-term user satisfaction may not align with short-term ranking accuracy, especially if users prioritize surprise or variety over time. Preferences that depend on context, such as time of day, mood, or social situation, cannot be assessed with this dataset. Additionally, new users present a challenge that this evaluation highlights but does not address. Reporting by user activity level would improve the analysis.

Implementation

All metrics are implemented in `src/evaluation/metrics.py`, which wraps the Microsoft recommenders library. Models must output predictions as DataFrames with standardized columns: `user_id`, `item_id`, and `prediction`. The evaluation functions accept test data containing ground-truth `rating` values and prediction DataFrames. They return metric dictionaries.

Conclusion

The temporal split using nDCG@k as the main metric gives a realistic way to evaluate recommendations while respecting their order. Secondary metrics add useful insights, and beyond-accuracy metrics help prevent poor solutions that trade diversity for accuracy. This method is applied consistently to all models described in the following sections, allowing for fair comparisons.